

Funcionamento dos Algoritmos da Mochila

Diogo Tadeu Campos

1 Backtracking

1.1 Funcionamento geral

O algoritmo de **Backtracking** resolve o problema explorando uma árvore de decisão. Para cada item da instância, existem duas escolhas possíveis: incluir o item na mochila ou não incluir. Assim, o algoritmo percorre recursivamente essas possibilidades até encontrar a melhor combinação.

A execução ocorre na seguinte ordem:

1. A função `main()` lê a instância da mochila.
2. É criado um objeto da estrutura `BacktrackingSolver`.
3. No construtor, os itens são ordenados por densidade normalizada.
4. Também é criado o vetor `suffixValue`, usado para estimar o valor restante.
5. A função `solve()` chama a busca recursiva `search()`.
6. A função `search()` testa incluir ou não incluir cada item.
7. Durante a busca, ramos inviáveis ou ruins são podados.
8. Ao final, o algoritmo retorna a melhor solução encontrada.

1.2 Estruturas de dados utilizadas

- `vector<Item> items`: armazena os itens da instância.
- `vector<long long> suffixValue`: guarda a soma dos valores dos itens restantes a partir de cada posição.
- `vector<int> currentChosen`: guarda os itens escolhidos no ramo atual da árvore.
- `vector<int> bestChosen`: guarda os itens da melhor solução encontrada.
- `long long bestValue`: armazena o maior lucro encontrado até o momento.

O vetor `suffixValue` é importante porque permite saber rapidamente qual é o maior valor que ainda poderia ser obtido em um ramo. Por exemplo, se o valor atual somado a todos os valores restantes ainda não supera `bestValue`, esse ramo pode ser descartado.

1.3 Funções mais importantes

Construtor BacktrackingSolver()

O construtor prepara os dados antes da busca. Ele ordena os itens por densidade normalizada, colocando primeiro aqueles que parecem mais vantajosos em relação ao peso e ao volume. Isso não muda a resposta ótima, mas pode ajudar o algoritmo a encontrar boas soluções mais cedo.

```
1 std::sort(items.begin(), items.end(), [&](const Item &a, const Item
  &b) {
2     return normalizedDensity(a, inst.capacityWeight, inst.
      capacityVolume)
3         > normalizedDensity(b, inst.capacityWeight, inst.
      capacityVolume);
4 });
```

Depois disso, o construtor calcula o vetor `suffixValue`.

```
1 for (int i = items.size() - 1; i >= 0; --i) {
2     suffixValue[i] = suffixValue[i + 1] + items[i].value;
3 }
```

Esse vetor é usado como uma estimativa simples do valor máximo restante.

Função `search()`

A função `search()` é a parte principal do Backtracking. Ela recebe a posição atual, o peso acumulado, o volume acumulado e o lucro atual.

Primeiro, ela verifica se o ramo já ultrapassou os limites da mochila:

```
1 if (weight > inst.capacityWeight || volume > inst.capacityVolume) {
2     return;
3 }
```

Depois, aplica a poda pelo valor restante:

```
1 if (value + suffixValue[pos] <= bestValue) {
2     return;
3 }
```

Essa condição significa que, mesmo pegando todos os itens restantes, aquele ramo não conseguiria melhorar a solução atual.

Em seguida, o algoritmo gera duas chamadas recursivas: uma incluindo o item e outra ignorando o item.

```
1 currentChosen.push_back(item.index);
2 search(pos + 1, weight + item.weight, volume + item.volume, value +
  item.value);
3 currentChosen.pop_back();
4
5 search(pos + 1, weight, volume, value);
```

O uso de `push_back()` e `pop_back()` permite adicionar temporariamente um item à solução atual e removê-lo ao retornar da recursão.

Função `solve()`

A função `solve()` inicia a busca e mede o tempo de execução. Ao final, monta a estrutura de resposta com lucro, itens escolhidos e tempo gasto.

1.4 Complexidade

No pior caso, o algoritmo pode testar todas as combinações possíveis de itens. Como cada item possui duas possibilidades, a complexidade é:

$$O(2^n)$$

Porém, na prática, as podas reduzem a quantidade de ramos explorados.

1.5 Análise Assintótica

O Backtracking explora uma árvore de decisão binária, onde cada nível representa a decisão de incluir ou não um item na mochila. Como existem duas possibilidades para cada um dos n itens, o número máximo de nós da árvore é da ordem de

$$2^n.$$

Dessa forma, no pior caso, o algoritmo visita todos os nós da árvore, resultando em uma complexidade de

$$O(2^n).$$

Nesta implementação, entretanto, são aplicadas duas estratégias de poda. A primeira interrompe imediatamente um ramo quando o peso ou o volume ultrapassam a capacidade da mochila. A segunda utiliza o vetor `suffixValue` para estimar o maior lucro que ainda pode ser obtido naquele ramo. Caso essa estimativa seja menor ou igual ao melhor lucro encontrado até o momento, o ramo é descartado.

Essas podas não alteram a complexidade assintótica do pior caso, que continua sendo exponencial, porém reduzem significativamente o número de nós explorados na prática.

2 Programação Dinâmica

2.1 Funcionamento geral

A solução por **Programação Dinâmica** trabalha com estados. Cada estado representa uma combinação de peso e volume já utilizados, associada ao melhor lucro obtido naquela situação.

A execução ocorre da seguinte forma:

1. A função `solveDP()` cria o estado inicial da mochila vazia.
2. Os estados são guardados em um `unordered_map`.
3. Para cada item, o algoritmo percorre os estados existentes.
4. Para cada estado, tenta criar um novo estado adicionando o item atual.
5. Se o novo estado respeitar peso e volume, ele é armazenado.
6. Se já existir um estado com mesmo peso e volume, permanece o de maior lucro.
7. Periodicamente, estados dominados são removidos.
8. No final, o melhor estado é escolhido e os itens são reconstruídos.

2.2 Estruturas de dados utilizadas

`unordered_map`

O `unordered_map` é uma tabela hash da STL. Ele foi usado para armazenar os estados da programação dinâmica.

Diferente de uma matriz tradicional, que poderia ocupar muita memória, o `unordered_map` guarda apenas os estados realmente alcançados. Por isso, essa abordagem é chamada de programação dinâmica com **estados esparsos**.

Em média, as operações de busca, inserção e atualização no `unordered_map` custam:

$$O(1)$$

Isso é útil porque o algoritmo pode gerar muitos estados.

`State`

A estrutura `State` guarda as informações principais de cada estado:

- `value`: lucro obtido naquele estado.
- `node`: índice usado para reconstruir a solução.

```
1 struct State {  
2     long long value;  
3     int node;  
4 };
```

ParentNode

A estrutura `ParentNode` permite reconstruir quais itens foram escolhidos. Sempre que um novo estado é criado, o algoritmo guarda o item adicionado e o estado anterior.

```
1 struct ParentNode {  
2     int itemIndex;  
3     int previous;  
4 };
```

Ao final, o algoritmo percorre esses ponteiros de trás para frente para recuperar a solução.

`makeKey()`, `keyWeight()` e `keyVolume()`

Como o estado depende de dois valores, peso e volume, o algoritmo precisa transformar esse par em uma única chave para usar no `unordered_map`. Isso é feito pela função `makeKey()`.

```
1 static std::uint64_t makeKey(int weight, int volume) {  
2     return (static_cast<std::uint64_t>(weight) << 32) | volume;  
3 }
```

A ideia é colocar o peso nos 32 bits mais significativos e o volume nos 32 bits menos significativos. Depois, as funções `keyWeight()` e `keyVolume()` recuperam essas informações.

Fenwick Tree

A **Fenwick Tree**, também chamada de **Binary Indexed Tree**, é uma estrutura usada para fazer consultas rápidas em vetores.

Neste código, ela aparece na classe `FenwickMax`. Em vez de calcular somas, como no uso mais comum, ela foi adaptada para guardar o maior valor encontrado até certo volume.

Ela é usada dentro da função `pruneDominated()` para identificar estados dominados de forma mais eficiente.

Um estado é dominado quando existe outro estado que usa menos ou igual peso, menos ou igual volume e obtém valor maior ou igual. Nesse caso, o estado dominado nunca será melhor e pode ser removido.

2.3 Funções mais importantes

Função `solveDP()`

É a função principal da programação dinâmica. Ela cria o mapa de estados, inicializa a mochila vazia e percorre os itens da instância.

```
1 states.emplace(makeKey(0, 0), State{0, 0});
```

Para cada item, o algoritmo cria uma cópia dos estados atuais. Essa cópia é necessária para evitar que um item seja usado mais de uma vez na mesma etapa.

```
1 for (const auto &kv : states) {  
2     snapshot.push_back(kv);  
3 }
```

Depois, tenta adicionar o item atual a cada estado existente.

```
1 const int weight = keyWeight(kv.first) + item.weight;  
2 const int volume = keyVolume(kv.first) + item.volume;
```

Se o novo peso e volume forem válidos, o novo estado é criado ou atualizado.

Função `pruneDominated()`

Essa função reduz o número de estados guardados. Ela percorre os estados e remove aqueles que são dominados por outros melhores.

Isso é importante porque, sem essa remoção, a quantidade de estados pode crescer muito rapidamente, aumentando o consumo de memória e o tempo de execução.

Classe `FenwickMax`

A classe `FenwickMax` possui duas operações principais:

- `update()`: atualiza o melhor valor conhecido em uma posição.
- `query()`: consulta o maior valor acumulado até certo índice.

Essas operações custam:

$$O(\log V)$$

onde V é a capacidade máxima de volume.

2.4 Complexidade

Em uma programação dinâmica tradicional com peso e volume, o pior caso seria proporcional a:

$$O(n \cdot W \cdot V)$$

onde n é o número de itens, W é a capacidade de peso e V é a capacidade de volume.

Neste código, como os estados são esparsos, a complexidade depende da quantidade real de estados gerados. A remoção de estados dominados ajuda a reduzir bastante esse número na prática.

2.5 Análise Assintótica

Na programação dinâmica, a complexidade depende da quantidade de estados gerados durante a execução.

Em uma implementação tradicional para a mochila com duas restrições, seria construída uma tabela tridimensional contendo todos os itens, pesos e volumes possíveis, resultando em uma complexidade

$$O(nWV),$$

onde n representa a quantidade de itens, W a capacidade máxima de peso e V a capacidade máxima de volume.

Nesta implementação, entretanto, utiliza-se um `unordered_map` para armazenar apenas os estados realmente alcançados durante a execução, caracterizando uma programação dinâmica por estados esparsos.

Além disso, a função `pruneDominated()` remove estados dominados utilizando uma Fenwick Tree, evitando que estados inferiores continuem ocupando memória e sendo processados.

Assim, embora o pior caso ainda seja proporcional ao número total de estados possíveis, na prática o algoritmo processa uma quantidade muito menor de estados, reduzindo significativamente o tempo de execução e o consumo de memória.

3 Branch and Bound

3.1 Funcionamento geral

O algoritmo de **Branch and Bound** também explora uma árvore de decisão, assim como o Backtracking. A diferença é que ele usa limites superiores para decidir se vale a pena continuar explorando um ramo.

A execução segue esta ordem:

1. A função `main()` lê a instância.
2. É criado um objeto `BranchAndBoundSolver`.
3. O construtor ordena os itens e cria estruturas auxiliares.
4. A função `greedyInitialSolution()` cria uma solução inicial gulosa.
5. A função `solve()` inicia a busca recursiva.
6. A função `search()` explora a árvore de decisão.
7. Antes de explorar um ramo, `upperBound()` calcula um limite superior.
8. Se o limite não superar a melhor solução, o ramo é podado.

3.2 Estruturas de dados utilizadas

- `vector<Item> items`: itens ordenados pela densidade normalizada.
- `vector<BBItem> weightOrder`: itens ordenados pela razão valor/peso.
- `vector<BBItem> volumeOrder`: itens ordenados pela razão valor/volume.
- `suffixValue`: soma dos valores restantes.
- `currentChosen`: solução atual durante a busca.
- `bestChosen`: melhor solução encontrada.

As listas `weightOrder` e `volumeOrder` são usadas para calcular limites superiores considerando separadamente as restrições de peso e volume.

3.3 Conceitos importantes

Solução gulosa inicial

Antes da busca principal, o algoritmo monta uma solução inicial usando uma estratégia gulosa. Ele percorre os itens ordenados e adiciona aqueles que cabem na mochila.

Essa solução pode não ser ótima, mas fornece um valor inicial para `bestValue`. Quanto maior esse valor inicial, mais ramos podem ser podados depois.

Upper Bound

O *upper bound*, ou limite superior, é uma estimativa otimista do maior lucro que ainda poderia ser obtido em um ramo.

Se esse limite for menor ou igual ao melhor lucro já encontrado, significa que aquele ramo não tem chance de produzir uma solução melhor. Portanto, ele pode ser descartado.

Relaxação fracionária

Para calcular alguns limites, o algoritmo considera uma versão relaxada do problema, permitindo que parte de um item seja utilizada. Isso não é permitido na solução final da mochila 0-1, mas serve apenas para estimar um valor máximo otimista.

Essa estimativa ajuda na poda porque, se nem em uma situação mais favorável o ramo supera a melhor solução, ele certamente não superaria no problema real.

3.4 Funções mais importantes

Construtor BranchAndBoundSolver()

O construtor prepara as estruturas auxiliares. Primeiro, ordena os itens por densidade normalizada. Depois, cria `suffixValue`, `weightOrder` e `volumeOrder`.

```
1 std::sort(weightOrder.begin(), weightOrder.end(), [](const BBIItem &  
2   a, const BBIItem &b) {  
3     return static_cast<double>(a.item.value) / a.item.weight  
4     > static_cast<double>(b.item.value) / b.item.weight;  
5   });
```

Essa ordenação auxilia no cálculo do limite considerando a restrição de peso.

Função greedyInitialSolution()

Essa função cria uma solução inicial simples. Ela percorre os itens e adiciona o item se ele couber na mochila.

```
1 if (weight + item.weight <= inst.capacityWeight &&  
2     volume + item.volume <= inst.capacityVolume) {  
3     weight += item.weight;  
4     volume += item.volume;  
5     value += item.value;  
6     chosen.push_back(item.index);  
7 }
```

O objetivo é começar a busca com um `bestValue` razoável, aumentando o número de podas.

Funções boundByWeight() e boundByVolume()

A função `boundByWeight()` calcula uma estimativa otimista considerando apenas a capacidade de peso. Já `boundByVolume()` faz o mesmo considerando apenas a capacidade de volume.

Essas funções usam a ideia da mochila fracionária: se um item não cabe inteiro, uma fração dele pode ser considerada apenas no cálculo do limite.

```
1 bound += static_cast<double>(item.value) * capacity / item.weight;
```

Esse trecho representa a adição proporcional de parte do item ao limite.

Função upperBound()

A função `upperBound()` combina três limites:

- valor atual mais `suffixValue`;

- limite considerando peso;
- limite considerando volume.

O algoritmo usa o menor desses valores, pois ele é o limite mais restritivo.

```
1 return std::min(byRemainingValue, std::min(byWeight, byVolume));
```

Função `search()`

A função `search()` realiza a exploração da árvore. Antes de continuar em um ramo, ela calcula o limite superior:

```
1 if (upperBound(pos, weight, volume, value) <= bestValue) {
2     return;
3 }
```

Se o limite não supera a melhor solução, o ramo é descartado.

Depois disso, o algoritmo tenta incluir o item e, em seguida, tenta ignorá-lo, como no Backtracking.

3.5 Complexidade

No pior caso, o Branch and Bound também pode explorar todas as combinações possíveis, resultando em:

$$O(2^n)$$

Entretanto, na prática, ele tende a ser mais eficiente que o Backtracking simples, pois usa uma solução inicial gulosa e limites superiores para eliminar muitos ramos da árvore.

3.6 Análise Assintótica

Assim como o Backtracking, o Branch and Bound percorre uma árvore binária de decisões, onde cada item pode ser incluído ou não na mochila.

No pior caso, quando nenhuma poda é realizada, toda a árvore precisa ser explorada, resultando em uma complexidade

$$O(2^n).$$

A principal diferença desta abordagem está na utilização de limites superiores (*Upper Bound*) para eliminar ramos que certamente não produzirão uma solução melhor que a atual.

Nesta implementação, o limite superior é calculado pelas funções `boundByWeight()` e `boundByVolume()`, além do vetor `suffixValue`. Também é construída uma solução gulosa inicial por meio da função `greedyInitialSolution()`, permitindo que o algoritmo encontre rapidamente um bom valor inicial para `bestValue`.

Quanto melhor for essa solução inicial, maior será a quantidade de ramos descartados ao longo da busca.

Embora essas estratégias reduzam drasticamente o número de nós visitados na prática, elas não modificam a complexidade do pior caso, que permanece exponencial.